Seta Planner Tools — Setup & Testing Guide

MS365 Planner CRUD via `POST /v1/tools/invoke` · dev environment only

1 · Prerequisites

Azure AD App Registration (portal.azure.com)

1. Open **Azure Active Directory → App registrations → New registration**
2. Set **Redirect URI** to: `http://localhost:8080/oauth/entra/callback`
3. Under **Expose an API**, add scope `access_as_user`
4. Under **API permissions → Add a permission → Microsoft Graph**, add:

Permission	Type	Admin consent
`Tasks.ReadWrite`	Delegated	No (but grant it)
`Group.Read.All`	Delegated	Yes
`Group.ReadWrite.All`	Delegated	Yes
`Group.Read.All`	Application	Yes
`Tasks.Read.All`	Application	Yes

5. Click **Grant admin consent for <your org>**
6. Copy **Application (client) ID** and create a **Client secret**

2 · Environment Setup

Create `apps/api/.env` (or root `.env`). Minimum required vars:

```
NODE_ENV=development
PORT=8080
DATABASE_URL=postgres://seta:dev@localhost:5432/seta
PUBLIC_BASE_URL=http://localhost:8080

# Azure AD app
ENTRA_CLIENT_ID=<Application (client) ID>
ENTRA_CLIENT_SECRET=<Client secret value>

# KMS — use env-based DEK for local dev
KMS_PROVIDER=env
DEV_DEK_BASE64=<base64 of 32 random bytes>
# generate: openssl rand -base64 32

# HMAC key for continuation tokens (≥ 64 hex chars = 32 bytes)
CONTINUATION_HMAC_KEY=<output of: openssl rand -hex 32>

# Planner cache TTLs (defaults shown)
PLANNER_CACHE_TTL_TASKS_SEC=60
PLANNER_CACHE_TTL_PLANS_SEC=600
PLANNER_CACHE_TTL_BUCKETS_SEC=300
PLANNER_CACHE_STALE_FALLBACK_MAX_SEC=3600
PLANNER_BATCH_CONCURRENCY=3
CONTINUATION_TTL_MIN=15

# LLM adapters — optional; omit any provider you don't have
# ANTHROPIC_API_KEY=sk-ant-...
# OPENAI_API_KEY=sk-...
```

Generate secrets

```
# DEK (32 bytes → base64)
openssl rand -base64 32

# HMAC key (32 bytes → 64 hex chars)
openssl rand -hex 32
```

Start the database & API

```
pnpm db:up          # start Postgres in Docker
pnpm migrate        # apply all migrations
nvm use 22          # pnpm v11 requires Node ≥ 22
pnpm --filter @seta/api dev
```

3 · OAuth Consent Flow

> This grants the app permission to act on your tenant and stores an app-level token in the vault.

1 **Get the admin-consent URL**

```
POST http://localhost:8080/oauth/entra/consent-url
Content-Type: application/json

{
  "connectors": ["ms365-planner", "ms365-directory"],
  "tenantHint": "<your-entra-tenant-id>"
}
```

Response: `{ "url": "https://login.microsoftonline.com/...", "state": "..." }`

2. **Open the URL in a browser**
Sign in as a Global Admin and approve the consent. You will be redirected back to `/oauth/entra/callback` and see: *"Connected — Your team can now @ mention SetaAgent in Microsoft Teams."*

3. **Verify in DB**

```
SELECT tenant_id, connector_id, status
FROM tenant.tenant_connectors
WHERE tenant_id = '<your-tenant-id>';
```

Expected: two rows with `status = active` for `ms365-planner` and `ms365-directory`.

4 · Get a User Access Token (OBO Exchange)

> The planner tools call Graph on behalf of the user. You need a delegated token in the vault.

1. **Obtain a user access token for your app's API**
Use the Azure CLI (easiest for dev):

```
az login --tenant <your-tenant-id>
az account get-access-token \
  --resource api://<ENTRA_CLIENT_ID> \
  --query accessToken -o tsv
```

Or use the MSAL browser flow against your app's `/authorize` endpoint with scope `api://<client-id>/access_as_user`.

2. **Exchange for an OBO (Graph) token**

```
POST http://localhost:8080/oauth/entra/exchange-obo
Content-Type: application/json

{
  "tenantId": "<your-tenant-id>",
  "userAssertion": "<access-token-from-step-1>",
  "scopes": [
    "https://graph.microsoft.com/Tasks.ReadWrite",
    "https://graph.microsoft.com/Group.Read.All"
  ]
}
```

Response: `{ "ok": true, "homeAccountId": "<objectId>.<tenantId>" }`

3 **Verify & capture your User ID**

```
SELECT provider_id, partition_key, expires_at
FROM oauth.oauth_tokens
WHERE tenant_id = '<your-tenant-id>';
```

You should see `partition_key = user:<objectId>.<tenantId>`.
Your `X-User-Id` for all tool calls = the full `homeAccountId` value (e.g. `5760fc90-...7c77.d7f9f1a0-...424b`).

5 · Postman Request Template

All tool invocations use the same endpoint and headers:

Field	Value
Method	`POST`
URL	`http://localhost:8080/v1/tools/invoke`
`Content-Type`	`application/json`
`X-Tenant-Id`	Your Entra tenant ID
`X-User-Id`	Full `homeAccountId` from step 4

> This route is **dev-only**. It is disabled when `NODE_ENV=production`.

To list all available tool IDs, send an unknown tool name:

```
{ "tool": "?", "input": {} }
```

6 · Read Tools

Read tools never mutate data. Run these first to collect real IDs for write operations.

`planner.list_my_tasks`

```
{ "tool": "planner.list_my_tasks", "input": {} }
```

Returns all tasks assigned to the current user across all plans. Use to get real `taskId` and `planId` values.

`planner.list_plans`

```
{ "tool": "planner.list_plans", "input": {} }
```

Lists all Planner plans the user has access to. Returns `planId` and `ownerGroupId` per plan.

`planner.list_plan_tasks`

```
{ "tool": "planner.list_plan_tasks", "input": { "planId": "<planId>" } }
```

`planner.list_buckets`

```
{ "tool": "planner.list_buckets", "input": { "planId": "<planId>" } }
```

Returns buckets. Use to get a real `bucketId` for task creation.

`planner.get_task`

```
{ "tool": "planner.get_task", "input": { "taskId": "<taskId>" } }
```

Returns task + details (description, checklist, references). Served from cache if fresh.

`planner.workload_analysis`

```
{ "tool": "planner.workload_analysis", "input": { "scope": { "planId": "<planId>" } } }
```

Returns per-assignee task counts + overdue counts, plus chart-ready series data for a bar chart.

7 · Write Tools — Preview → Commit Flow

Every write is two calls: **preview** returns a signed `token` (valid 15 min, single-use); **commit** consumes it and performs the actual Graph mutations.

Update Tasks

```
// Preview
{ "tool": "planner.update_tasks.preview", "input": {
    "updates": [
      { "taskId": "<taskId>", "title": "New title", "priority": 5 },
      { "taskId": "<taskId2>", "percentComplete": 50, "dueDateTime": "2026-06-01T00:00:00Z"
}
    ]
}}

// Commit
{ "tool": "planner.update_tasks.commit", "input": { "token": "<token from preview>" } }
```

Updatable fields: `title`, `priority` (0–10), `percentComplete` (0–100), `dueDateTime` (ISO 8601 or null), `assignees` (array of user IDs), `bucketId`, `appliedCategories`.

Complete Tasks

```
{ "tool": "planner.complete_tasks.preview", "input": {
    "taskIds": ["<taskId>", "<taskId2>"]
}}
{ "tool": "planner.complete_tasks.commit", "input": { "token": "<token>" } }
```

Create Tasks

```
{ "tool": "planner.create_tasks.preview", "input": {
    "tasks": [{
      "planId": "<planId>",
      "title": "New task from Seta",
      "bucketId": "<bucketId>",         // optional
      "assignees": ["<userId>"],         // optional — Entra object IDs
      "dueDateTime": "2026-06-15T00:00:00Z", // optional
      "priority": 3                          // optional, 0–10
    }]
}}
{ "tool": "planner.create_tasks.commit", "input": { "token": "<token>" } }
```

Add Comments

```
{ "tool": "planner.add_comments.preview", "input": {
    "comments": [
      { "taskId": "<taskId>", "content": "Comment from Seta agent" }
    ]
}}
{ "tool": "planner.add_comments.commit", "input": { "token": "<token>" } }
```

Create Plan

> Requires `Group.Read.All` delegated scope in the OBO token.

```
{ "tool": "planner.create_plan.preview", "input": {
    "ownerGroupId": "<M365-group-id>",
    "title": "Seta Test Plan"
}}
{ "tool": "planner.create_plan.commit", "input": { "token": "<token>" } }
```

8 · DB Verification Queries

```
-- Cache populated after reads
SELECT task_id, title, etag, updated_at
FROM connector_ms365_planner.planner_tasks_cache
WHERE tenant_id = '<your-tenant-id>'
ORDER BY updated_at DESC LIMIT 10;

-- Continuation tokens (consumed_at fills in after commit)
SELECT id, tool_id, created_at, expires_at, consumed_at
FROM agent.write_continuations
WHERE tenant_id = '<your-tenant-id>'
ORDER BY created_at DESC LIMIT 10;

-- Every Graph call audited
SELECT operation, result, created_at
FROM audit.audit_log
WHERE tenant_id = '<your-tenant-id>'
ORDER BY created_at DESC LIMIT 20;

-- Active connector status
SELECT connector_id, status, consented_at
FROM tenant.tenant_connectors
WHERE tenant_id = '<your-tenant-id>';

-- Stored tokens (app + user)
SELECT provider_id, partition_key, expires_at
FROM oauth.oauth_tokens
WHERE tenant_id = '<your-tenant-id>';
```

9 · Common Errors

Error	Cause	Fix
`no token for user`	Vault has no `user:<homeAccountId>` entry	Redo the `exchange-obo` call (§4). Use the full `homeAccountId` as `X-User-Id`.
`AADSTS65001 consent_required`	Delegated permissions not registered or not consented in Azure AD	Add delegated `Tasks.ReadWrite` + `Group.Read.All` to app registration and grant admin consent.
`consent required for connector`	Tenant not in `tenant_connectors`	Redo OAuth consent flow (§3).

`continuation signature invalid`	Token was tampered with or `CONTINUATION_HMAC_KEY` changed	Redo the preview call to get a fresh token.
`continuation expired`	Token older than 15 minutes	Redo the preview call.
`continuation already consumed`	Commit called twice with the same token	Tokens are single-use. Redo preview for a new one.
`412 Conflict` in commit result	ETag mismatch — task was modified by someone else between preview and commit	Normal behaviour. Redo preview (re-fetches latest ETag) then commit again.

Generated 2026-05-12 · Seta OS · feat/ms365-planner-crud · dev environment only